CHECKPOINT 2 · ATIVIDADE PRÁTICA

ESCALA COMPUTACIONAL EM REDES NEURAIS

IA e Machine Learning · Engenharia de Software | ANN em scikit-learn e PyTorch · CPU × GPU · custo × desempenho

Turma	Valor	Integrante(s)	Data
	10	Lucca Borges — RM 554608 Ruan Melo — RM 557599 Rodrigo Jimenez — RM 558148 João Victor Franco — RM 556790 Bruno Leão — RM 555563	01/10/2026

1. Contexto da atividade

Nesta atividade, vocês vão executar dois experimentos complementares com uma rede neural artificial aplicada à classificação de tumores benignos e malignos. O foco não é apenas buscar a maior acurácia. Primeiro, vocês observarão como o aumento da arquitetura altera parâmetros, tempo de treinamento e desempenho. Depois, executarão a mesma ANN em CPU e GPU para investigar quando o paralelismo começa a compensar. A atividade conecta treinamento de modelos, execução computacional e arquitetura de hardware, ampliando a compreensão sobre escalabilidade em IA e Machine Learning.

> **Ideia central**
> Troque uma variável de cada vez. No Experimento A, o hardware fica fixo e a rede cresce. No Experimento B, a rede e os dados ficam fixos e apenas o dispositivo muda: CPU → GPU.

2. Objetivos

- Executar o pipeline com 2 features e depois com 22 features.
- Comparar arquiteturas progressivamente maiores no scikit-learn e no PyTorch.
- Registrar parâmetros, tempo, iterações/épocas e acurácia produzidos no Colab.
- Avaliar se uma rede maior produz ganho proporcional ao custo computacional.
- Comparar a mesma implementação PyTorch em CPU e GPU e calcular o speedup.
- Relacionar a escala de treinamento à necessidade de GPUs, clusters e supercomputadores.
- Interpretar o custo-benefício do aumento da rede, sem assumir que uma arquitetura maior é automaticamente melhor.

3. Procedimento obrigatório

Etapa A — demonstração guiada com 2 features

- Use o recorte didático com radius_mean e texture_mean.
- Execute o experimento de escala com as arquiteturas definidas no notebook.
- Registre os valores reais da execução em CPU.

Etapa B — atividade autônoma com 22 features

- Troque a definição de X para: X = dados_reduzidos.drop(columns=['diagnosis']).
- Execute novamente a partir da célula de separação entre X e y, incluindo treino/teste, scaler, scikit-learn, PyTorch e experimento de escala.
- Compare 2 e 22 features usando os valores da sua própria execução.

Etapa C — mesma ANN em CPU × GPU

- No Colab, ative a GPU em: Ambiente de execução → Alterar tipo de ambiente de execução → Acelerador de hardware → GPU.
- Execute o bloco CPU × GPU mantendo dataset, features, arquitetura, épocas, função de perda e otimizador.
- Registre tempo em CPU, tempo em GPU e speedup = tempo_CPU / tempo_GPU.
- Se a GPU não estiver disponível, registre a indisponibilidade e execute a análise com o que foi possível medir.

Controle experimental

No Experimento A, compare arquiteturas dentro de cada framework. No Experimento B, compare CPU e GPU somente no PyTorch, mantendo a mesma rede. Isso evita misturar mudanças de software e hardware na mesma conclusão.

O que este experimento NÃO mede

Tempo de execução e número de parâmetros são indicadores de custo computacional. Eles não medem energia elétrica diretamente. Portanto, não conclua “menor tempo = menor energia” sem uma medição de potência/energia.

4. Registro experimental — Tabela A: escala da ANN em CPU

Preencha com valores realmente produzidos no seu Colab.

Entrada	Framework	Arquitetura	Parâmetros	Tempo (s)	Iter./épocas	Acurácia
2 features	scikit-learn (CPU)	(8,)	33	0,179	309	88,60%
2 features	scikit-learn (CPU)	(64,64)	4.417	0,190	128	89,47%
2 features	scikit-learn (CPU)	(128,128,128)	33.537	1,527	308	91,23%
2 features	scikit-learn (CPU)	(256,256,256)	132.609	1,816	135	91,23%
2 features	PyTorch (CPU)	(8,)	33	0,347	200	87,72%
2 features	PyTorch (CPU)	(64,64)	4.417	0,477	200	91,23%
2 features	PyTorch (CPU)	(128,128,128)	33.537	0,693	200	91,23%
2 features	PyTorch (CPU)	(256,256,256)	132.609	1,058	200	87,72%
22 features	scikit-learn (CPU)	(8,)	193	0,294	491	97,37%
22 features	scikit-learn (CPU)	(64,64)	5.697	0,257	158	97,37%
22 features	scikit-learn (CPU)	(128,128,128)	36.097	0,365	70	95,61%
22 features	scikit-learn (CPU)	(256,256,256)	137.729	0,818	57	95,61%
22 features	PyTorch (CPU)	(8,)	193	0,356	200	98,25%
22 features	PyTorch (CPU)	(64,64)	5.697	0,494	200	96,49%
22 features	PyTorch (CPU)	(128,128,128)	36.097	0,690	200	95,61%
22 features	PyTorch (CPU)	(256,256,256)	137.729	0,997	200	95,61%

Leitura esperada

Observe se o crescimento do número de parâmetros e do tempo é acompanhado por melhora proporcional da acurácia. Uma rede maior pode aumentar fortemente o custo sem melhorar o resultado na mesma proporção.

5. Registro experimental — Tabela B: PyTorch em CPU × GPU

Use a mesma ANN, os mesmos dados e o mesmo número de épocas. Mude apenas o dispositivo.

Entrada	Arquitetura	Parâmetros	CPU (s)	GPU (s)	Speedup	Acurácia GPU
2 features	(8,)	33	0,338	0,842	0,40	87,72%
2 features	(64,64)	4.417	0,470	1,023	0,46	91,23%
2 features	(128,128,128)	33.537	0,686	1,284	0,53	91,23%
2 features	(256,256,256)	132.609	1,057	1,325	0,80	87,72%
22 features	(8,)	193	0,342	0,855	0,40	98,25%
22 features	(64,64)	5.697	0,486	1,056	0,46	96,49%
22 features	(128,128,128)	36.097	0,717	1,228	0,58	95,61%
22 features	(256,256,256)	137.729	1,034	1,313	0,79	95,61%

Como interpretar o speedup

speedup > 1: a GPU foi mais rápida; speedup ≈ 1: tempos semelhantes; speedup < 1: a CPU foi mais rápida naquele caso. Para redes pequenas, a GPU pode não compensar o overhead de transferência, inicialização e coordenação.

Ponte para clusters e supercomputadores

Quando uma única GPU deixa de ser suficiente em cálculo ou memória, o treinamento pode ser distribuído entre várias GPUs e vários nós. Isso acrescenta outro custo: comunicação e movimentação de dados entre dispositivos.

6. Questões de análise — dependem do seu experimento

1. No seu run com 2 features, qual arquitetura apresentou a melhor relação entre tempo e acurácia? Cite os valores medidos.

2. Qual arquitetura teve o maior aumento de parâmetros sem produzir ganho proporcional de acurácia? Quantifique esse aumento.

3. Ao passar de 2 para 22 features, o que mudou em tempo e acurácia? Compare pelo menos duas arquiteturas e use números da sua tabela.

4. Escolha uma arquitetura e calcule a razão entre o número de parâmetros com 22 features e com 2 features. Explique de onde vem essa diferença.

5. No PyTorch, identifique no notebook as linhas que executam forward, cálculo da perda, backward e atualização dos pesos.

6. Os seus resultados permitem afirmar que uma rede maior é automaticamente melhor? Use pelo menos dois resultados experimentais para sustentar a resposta.

7. Na comparação CPU × GPU, em qual arquitetura apareceu o maior speedup? Informe tempo_CPU, tempo_GPU e o speedup calculado.

8. Alguma arquitetura pequena foi tão rápida ou mais rápida na CPU? Se ocorreu, proponha uma explicação relacionada ao overhead da GPU.

Regra de evidência

Nas questões 1, 2, 3, 6, 7 e 8 , use números obtidos no seu próprio Colab. Respostas genéricas que poderiam ser escritas sem executar o notebook não atendem ao objetivo da atividade.

Respostas da equipe às questões 1 a 8

Respostas redigidas sobre a execução registrada em 2026-10-14 08:16 (assinatura das dependências: f75e366e0320acdf). Todos os números vêm dessa execução — Tabelas A e B e seções 9 e 10. O teste tem 114 exames: 1 acerto equivale a 0,88 p.p.

1. No seu run com 2 features, qual arquitetura apresentou a melhor relação entre tempo e acurácia?

Resposta: a (64,64), nos dois frameworks — como melhor compromisso entre tempo e acurácia.

Critério: primeiro a fronteira de Pareto (arquiteturas para as quais nenhuma outra é ao mesmo tempo mais rápida e pelo menos tão precisa) — scikit-learn: (8,), (64,64) e (128,128,128); PyTorch: (8,) e (64,64). Dentro dela, a (64,64) é o melhor compromisso:

- **PyTorch:** a (64,64) atinge 91,23% em 0,477 s — a maior acurácia do PyTorch com 2 features. A (128,128,128) empata nessa acurácia gastando 0,693 s (+45% de tempo, 7,6× os parâmetros), e a (256,256,256) leva 1,058 s para cair a 87,72%. Em relação à (8,) (0,347 s; 87,72%), a (64,64) ganha +3,51 p.p. (4 acertos a mais) com +37% de tempo.
- **scikit-learn:** a (64,64) fica em 89,47% com 0,190 s, quase o tempo da (8,) (0,179 s; 88,60%). Para ganhar mais 1,75 p.p. (2 acertos), é preciso ir à (128,128,128) (1,527 s) ou à (256,256,256) (1,816 s): 8,1× e 9,6× o tempo da (64,64). Escolher a (64,64) aqui é um compromisso: abre mão desses 2 acertos em troca de um treino 8,1 vezes mais rápido.
- **Robustez (seção 10.1):** em 10 sorteios de treino/teste, a (64,64) tem a maior média do scikit-learn (91,32% ± 2,09 p.p.) e empata com a maior média do PyTorch (91,05% ± 1,79 p.p.); a vantagem pontual da (128,128,128) no scikit-learn não se repete na média (90,26% ± 1,82 p.p.).

2. Qual arquitetura teve o maior aumento de parâmetros sem produzir ganho proporcional de acurácia?

Resposta: a (256,256,256). Com 2 features ela tem 132.609 parâmetros — 4.018× os da (8,) (+132.576) e 3,95× os da (128,128,128) (+99.072). Em troca:

- **scikit-learn:** a mesma acurácia da (128,128,128) (91,23% → 91,23%, 0,00 p.p.) e só +2,63 p.p. sobre a (8,), com 10,1× o tempo da (8,) (1,816 s × 0,179 s).
- **PyTorch:** a acurácia **cai** de 91,23% na (128,128,128) para 87,72% (−3,51 p.p.) — o mesmo nível da (8,) —, com 3,05× o tempo da (8,).
- **Com 22 features** o padrão se repete: 714× os parâmetros da (8,) e −1,75 p.p. (scikit-learn) / −2,63 p.p. (PyTorch) de acurácia.

3. Ao passar de 2 para 22 features, o que mudou em tempo e acurácia?

- **Acurácia — subiu nas 8 combinações**, de +4,39 p.p. a +10,53 p.p. (exemplos: (8,) no PyTorch 87,72% → 98,25% (+10,53 p.p.); (8,) no scikit-learn 88,60% → 97,37% (+8,77 p.p.); (256,256,256) no scikit-learn 91,23% → 95,61% (+4,39 p.p.) e no PyTorch 87,72% → 95,61% (+7,89 p.p.)). As 20 medidas extras (concavidade, pontos côncavos, textura e suavidade no pior caso etc.) carregam muito mais informação sobre o tumor do que só o raio e a textura médios.
- **Tempo no PyTorch (200 épocas fixas) — praticamente não mudou** (variação de no máximo 6% entre as quatro arquiteturas): (8,) 0,347 s → 0,356 s (1,03×); (256,256,256) 1,058 s → 0,997 s (0,94×). Só a 1ª camada cresce (+3,9% de parâmetros na (256,256,256)), e nesse tamanho o tempo de cada passo pouco depende do número de entradas.
- **Tempo no scikit-learn (até atingir o critério de parada) — mudou pelo número de épocas:** a (8,) precisou de 491 épocas em vez de 309 e ficou mais lenta (0,179 s → 0,294 s); a (128,128,128) parou em 70 épocas em vez de 308 (1,527 s → 0,365 s, 4,2× mais rápida) e a (256,256,256) em 57 em vez de 135 (1,816 s → 0,818 s). Com 22 features, as redes maiores chegam antes ao platô da perda de treino que o critério de parada detecta — e todas, da (64,64) em diante, atingem 100% de acurácia no treino.

4. Razão entre o número de parâmetros com 22 e com 2 features — e de onde vem a diferença

(8,): 193 / 33 = **5,85**. **(256,256,256):** 137.729 / 132.609 = **1,039**.

Cada camada densa tem `n_entrada × n_saída` pesos + `n_saída` vieses, e só a 1ª camada depende do número de features. Passar de 2 para 22 features acrescenta (22 − 2) × n_1 pesos: 20 × 8 = 160 na (8,) — muito diante dos 33 parâmetros originais — e 20 × 256 = 5.120 na (256,256,256), pouco diante dos 131.072 pesos das duas camadas 256 × 256, que não dependem da entrada. Por isso a razão é grande nas redes pequenas e tende a 1 nas grandes.

5. Linhas que executam forward, cálculo da perda, backward e atualização dos pesos (PyTorch)

Na função `treinar_ann_pytorch` (seção 3 do notebook; a célula da seção 9 imprime as linhas com a numeração):

- linha 21: `otimizador.zero_grad()` — zera os gradientes acumulados no passo anterior;
- linha 22: `logits = modelo(X_train[lote])` — **forward** (a rede calcula os logits do lote);
- linha 23: `perda = criterio(logits, y_train[lote])` — **cálculo da perda** (`BCEWithLogitsLoss`);
- linha 24: `perda.backward()` — **backward** (retropropagação: gradiente da perda em relação a cada peso);
- linha 25: `otimizador.step()` — **atualização dos pesos** (passo do Adam).

Esse bloco roda uma vez por lote: 3 lotes × 200 épocas = 600 passos de otimização por treino.

6. Os resultados permitem afirmar que uma rede maior é automaticamente melhor?

Não. Cinco resultados desta execução contradizem a ideia:

- **22 features, scikit-learn:** a (8,) — 193 parâmetros, 0,294 s — acerta 97,37%, o mesmo que a (64,64) e mais que a (256,256,256) (137.729 parâmetros, 0,818 s), que fica em 95,61%.
- **22 features, PyTorch:** a (8,) tem a maior acurácia (98,25%); a (256,256,256) usa 714× mais parâmetros e 2,80× mais tempo para chegar a 95,61%.
- **2 features, PyTorch:** (256,256,256) = (8,) = 87,72%, abaixo da (64,64) (91,23%).
- **Sinal compatível com sobreajuste:** com 22 features, todas as redes a partir da (64,64) chegam a 100% de acurácia no treino, mas não passam de 97,37% no teste — a capacidade extra não se converteu em generalização.
- **Robustez (seção 10.1, comparação descritiva):** em 10 sorteios, as médias das quatro arquiteturas com 22 features ficam entre 95,96% e 96,75%, com desvio-padrão de 1,1 a 1,8 p.p. — a diferença entre arquiteturas é menor que a variação causada pelo próprio sorteio.

Aumentar a rede **não melhorou sistematicamente** a acurácia: houve ganhos pontuais (por exemplo, +3,51 p.p. da (8,) para a (64,64) no PyTorch com 2 features), mas nenhum que acompanhe o crescimento dos parâmetros. O que melhorou o resultado de forma consistente foi a informação de entrada (2 → 22 features: +4,39 p.p. a +10,53 p.p.).

7. Em qual arquitetura apareceu o maior speedup?

Na (256,256,256). Com 2 features: tempo_CPU = 1,057 s, tempo_GPU = 1,325 s, speedup = 1,057 / 1,325 = **0,80**. Com 22 features: 1,034 s / 1,313 s = **0,79**.

Mesmo no melhor caso o speedup ficou abaixo de 1 — a GPU levou cerca de 1,3× o tempo da CPU —, mas ele cresce a cada aumento da rede: 0,40 → 0,46 → 0,53 → 0,80 (2 features) e 0,40 → 0,46 → 0,58 → 0,79 (22 features).

8. Alguma arquitetura pequena foi tão rápida ou mais rápida na CPU?

Sim — e não só as pequenas: a CPU foi mais rápida nas 8 configurações. O critério é o das repetições: em cada configuração, até a execução mais lenta da CPU foi mais rápida que a execução mais rápida da GPU (caso mais apertado: 22 features (256,256,256), 1,115 s × 1,269 s). Na (8,) a CPU foi 2,5× mais rápida com 2 features (0,338 s × 0,842 s, speedup 0,40) e 2,5× com 22 features (0,342 s × 0,855 s, speedup 0,40).

Explicação compatível com os dados — o overhead da GPU domina quando há pouco cálculo por passo:

- **Muitas operações pequenas por passo (seção 10.3, `torch.profiler`, 22 features, lote 200):** cada passo de treino dispara, em média, 31 kernels CUDA na (8,) e 44 na (256,256,256) (forward, perda, backward e Adam). Na (8,), as durações desses kernels somam 0,062 ms por passo, para um passo de 1,79 ms de parede na mesma configuração (medido sem profiler); na (64,64), 0,085 ms para 2,34 ms. (Esse passo é mais lento que o da Tabela B porque aqui a preparação de cada treino se dilui em só 30 passos, e não em 600.) Nas redes pequenas, o cálculo na GPU é uma parte pequena do passo; o restante vai para lançar kernels, sincronizar e coordenar a partir da CPU — na (8,), a multiplicação é de uma matriz 200 × 22 por outra 22 × 8, pouco trabalho para os milhares de núcleos da GPU. Na (256,256,256) a soma sobe para 0,394 ms (para 2,18 ms de parede): há mais cálculo por passo, e é também onde o speedup é maior.
- **O tempo da GPU quase não acompanha o tamanho da rede:** da (8,) para a (256,256,256), com 2 features, os parâmetros crescem 4.018×, o tempo da CPU 3,1× (0,338 s → 1,057 s) e o da GPU só 1,6× (0,842 s → 1,325 s). Em tempo total amortizado por passo, a (8,) leva 1,40 ms na GPU contra 0,56 ms na CPU.
- **Transferência e inicialização:** o tempo da GPU inclui copiar modelo e dados para a placa (e o índice dos lotes a cada época). Além disso, o 1° treino "a frio" levou 1,435 s contra 0,842 s depois de aquecido: uma diferença observada de 0,592 s, compatível com custos de primeira execução, que pesam mais quanto mais curto é o treino.
- **Quando passa a compensar (seção 10.2):** com mais trabalho por passo a vantagem se inverte. Na (256,256,256), o lote completo (455 amostras) dá o maior speedup entre os lotes. Ali as duas medições da mesma configuração caem em lados opostos de 1 (1,43 na tabela por lote e 0,94 na tabela por largura): sem vencedor claro dentro do ruído de medição. Na (8,), o speedup fica abaixo de 0,6 em todos os lotes (0,39 a 0,53): o passo continua pequeno demais. Com camadas mais largas a GPU passa a ganhar com folga: speedup de 2,23 (512 neurônios por camada), 7,18 (1024) e 12,12 (2048). O custo da GPU por época quase não muda até 1024 neurônios (2,41 → 2,67 ms), enquanto o da CPU cresce 8,5× (2,26 → 19,18 ms).

Conclusão — custo × benefício

- **Escolha recomendada: 22 features com uma rede pequena — (8,) ou (64,64) — treinada em CPU.** Com 22 features elas acertam de 96,49% a 98,25% em 0,257 s a 0,494 s, enquanto a (256,256,256) usa 714× os parâmetros da (8,) e 2,8× o tempo sem acertar mais.
- **GPU:** neste problema (569 exames, até 137.729 parâmetros) a GPU não compensou — speedup de 0,40 a 0,80 —, o que é compatível com um trabalho por passo pequeno demais para amortizar lançamento de kernels, sincronização e transferência. Ela passou a valer a pena quando cada passo teve muito cálculo: 2,2× a 12,1× nas redes de 0,5 a 8,4 milhões de parâmetros com lote completo.
- **Ponte para clusters e supercomputadores:** a mesma lógica vale um nível acima. Quando o modelo ou os dados não cabem mais em uma GPU (em cálculo ou memória), o treino é distribuído entre várias GPUs e vários nós — e aparece outro custo, a comunicação (sincronizar gradientes e mover dados entre dispositivos). Distribuir só compensa quando o cálculo de cada dispositivo é grande o bastante para amortizar essa comunicação, do mesmo jeito que aqui a GPU só compensou com muito trabalho por passo.
- **O que não medimos:** tempo e número de parâmetros indicam custo computacional, não energia. Sem medir potência, não concluímos que "menor tempo = menor energia".
- **Ambiente:** a execução registrada foi local (Intel Core i7-11700K @ 3.60GHz, 8 núcleos/16 threads; NVIDIA GeForce RTX 3060 12 GB; Windows 11). Em outro hardware os tempos absolutos mudam; o notebook recalcula tabelas e evidências, e a checagem da seção 11 indica o que precisaria ser atualizado no texto.

7. Entrega

- Tabela A preenchida com valores reais do Colab.
- Tabela B CPU × GPU preenchida quando houver GPU disponível.
- Respostas às questões 1 a 8.
- Print ou cópia da saída do Colab contendo pelo menos uma tabela de escala e a comparação CPU × GPU.
- Identificação do(s) integrante(s).
-

8. Critérios de avaliação

Critério	O que será observado
Execução do experimento	Resultados reais, consistentes e coerentes com as arquiteturas executadas.
Análise quantitativa	Comparações sustentadas por parâmetros, tempo, speedup e acurácia.
Interpretação computacional	Relação correta entre crescimento da rede, custo, GPU, comunicação e necessidade de escala.
Custo-benefício do modelo	Decisão fundamentada sobre o tamanho da arquitetura, sem assumir que redes maiores produzem ganhos automáticos.
Clareza técnica	Texto objetivo, organizado e sem respostas genéricas.

Importante

O objetivo não é provar que “GPU é sempre melhor” nem que “uma rede maior é sempre melhor”. O objetivo é usar evidência experimental para identificar quando o custo cresce, quando o paralelismo ajuda e qual configuração oferece a melhor relação entre desempenho e custo computacional.

Anexo — cópia da saída do notebook executado

Notebook: CP2_Escala_Computacional_Redes_Neurais.ipynb · repositório github.com/lucksza/CP5-ML · execução de 2026-10-14 08:16.

Dados: `dados_reduzidos` (22 features) foi construído por um filtro de redundância — de cada par com |correlação| > 0,92 sai a segunda coluna —, calculado só nas 455 linhas de treino; o limiar foi escolhido por levar exatamente às 22 features, e a equivalência com a lista do notebook da disciplina não foi verificada. Removidas: perimeter_mean, area_mean, concave points_mean, perimeter_se, area_se, radius_worst, perimeter_worst, area_worst.

Ambiente da execução

```
       data_execucao: 2026-10-14 08:16
             sistema: Windows 11 (build 26200)
              python: 3.11.9
                 cpu: 11th Gen Intel(R) Core(TM) i7-11700K @ 3.60GHz
     nucleos_fisicos: 8
     nucleos_logicos: 16
              ram_gb: 31.9
 threads_pytorch_cpu: 8
               numpy: 2.4.6
              pandas: 3.0.6
        scikit-learn: 1.9.1
               torch: 2.14.1+cu130
      gpu_disponivel: True
```

Tabela A — escala da ANN em CPU (saída do notebook)

	Entrada	Framework	Arquitetura	Parâmetros	Tempo (s)	Iter./épocas	Acurácia
0	2 features	scikit-learn (CPU)	(8,)	33	0.179	309	88.60%
1	2 features	scikit-learn (CPU)	(64,64)	4,417	0.190	128	89.47%
2	2 features	scikit-learn (CPU)	(128,128,128)	33,537	1.527	308	91.23%
3	2 features	scikit-learn (CPU)	(256,256,256)	132,609	1.816	135	91.23%
4	2 features	PyTorch (CPU)	(8,)	33	0.347	200	87.72%
5	2 features	PyTorch (CPU)	(64,64)	4,417	0.477	200	91.23%
6	2 features	PyTorch (CPU)	(128,128,128)	33,537	0.693	200	91.23%
7	2 features	PyTorch (CPU)	(256,256,256)	132,609	1.058	200	87.72%
8	22 features	scikit-learn (CPU)	(8,)	193	0.294	491	97.37%
9	22 features	scikit-learn (CPU)	(64,64)	5,697	0.257	158	97.37%
10	22 features	scikit-learn (CPU)	(128,128,128)	36,097	0.365	70	95.61%
11	22 features	scikit-learn (CPU)	(256,256,256)	137,729	0.818	57	95.61%
12	22 features	PyTorch (CPU)	(8,)	193	0.356	200	98.25%
13	22 features	PyTorch (CPU)	(64,64)	5,697	0.494	200	96.49%
14	22 features	PyTorch (CPU)	(128,128,128)	36,097	0.690	200	95.61%
15	22 features	PyTorch (CPU)	(256,256,256)	137,729	0.997	200	95.61%

Tabela B — CPU × GPU e evidências das questões 7 e 8 (saída do notebook)

```
GPU: NVIDIA GeForce RTX 3060 (12.0 GB) | CUDA 13.0
criação do contexto CUDA (1ª operação na GPU): 0.029 s
1º treino da ANN (8,) com 2 features na GPU, ainda "fria": 1.435 s


Q7: maior speedup = 0.80 em (256,256,256) com 2 features (CPU 1.057 s / GPU 1.325 s)
Q8: leitura das faixas de 5 repetições em cada configuração: {'CPU mais rápida': 8}
    caso mais apertado: 22 features (256,256,256) — CPU máx. 1.115 s x GPU mín. 1.269 s
```

	Entrada	Arquitetura	Parâmetros	CPU (s)	GPU (s)	Speedup	Acurácia GPU	Leitura
0	2 features	(8,)	33	0.338	0.842	0.40	87.72%	CPU mais rápida
1	2 features	(64,64)	4,417	0.470	1.023	0.46	91.23%	CPU mais rápida
2	2 features	(128,128,128)	33,537	0.686	1.284	0.53	91.23%	CPU mais rápida
3	2 features	(256,256,256)	132,609	1.057	1.325	0.80	87.72%	CPU mais rápida

```
4  22 features           (8,)        193   0.342   0.855    0.40       98.25%  CPU mais rápida
5  22 features        (64,64)      5,697   0.486   1.056    0.46       96.49%  CPU mais rápida
6  22 features  (128,128,128)     36,097   0.717   1.228    0.58       95.61%  CPU mais rápida
7  22 features  (256,256,256)    137,729   1.034   1.313    0.79       95.61%  CPU mais rápida
2 features: parâmetros x4,018 da (8,) para a (256,256,256) | tempo CPU x3.13 | tempo GPU x1.57
22 features: parâmetros x714 da (8,) para a (256,256,256) | tempo CPU x3.02 | tempo GPU x1.54
acurácia idêntica em CPU e GPU em 8 de 8 configurações
```

Checagem das respostas (seção 11 do notebook)

```
(a) Interpretação — 24 de 24 afirmações das respostas continuam válidas nesta execução:
   ✔ Q1: PyTorch/2f: a (64,64) tem a maior acurácia, e a (128,128,128) empata com ela gastando mais tempo
   ✔ Q1: PyTorch/2f: a (256,256,256) é mais lenta que a (128,128,128) e menos precisa que a (64,64); a (64,64) é mais precisa que a (8,)
   ✔ Q1: scikit-learn/2f: a (64,64) leva menos de 1,3× o tempo da (8,); a (128,128,128) e a (256,256,256) acertam mais e levam mais de 5× o tempo da (64,64)
   ✔ Q1: fronteiras de Pareto com 2 features iguais às citadas: scikit-learn ['(8,)', '(64,64)', '(128,128,128)'], PyTorch ['(8,)', '(64,64)']
   ✔ Q1: robustez/2f: a (64,64) tem a maior média no scikit-learn e empata com a maior no PyTorch; a (128,128,128) fica abaixo dela no scikit-learn
   ✔ Q2: (256,256,256): mesma acurácia da (128,128,128) no scikit-learn/2f; abaixo da (128,128,128) e igual à (8,) no PyTorch/2f; abaixo da (8,) com 22 features nos dois frameworks
   ✔ Q3: a acurácia sobe de 2 para 22 features nas 8 combinações
   ✔ Q3: PyTorch: o tempo com 22 features fica entre 0,8× e 1,25× o tempo com 2 features em todas as arquiteturas
   ✔ Q3: scikit-learn: com 22 features a (8,) usa mais épocas e mais tempo; a (128,128,128) e a (256,256,256), menos épocas e menos tempo
   ✔ Q4: parâmetros da Tabela A = Σ (n_entrada × n_saída + n_saída) em todas as linhas
   ✔ Q5: linhas do treino citadas na Q5 continuam as mesmas (21, 22, 23, 24, 25)
   ✔ Q6: 22 features: no scikit-learn a (8,) empata com a (64,64) e supera a (256,256,256); no PyTorch a (8,) tem a maior acurácia
   ✔ Q6: 22 features: acurácia de treino de 100% da (64,64) em diante nos dois frameworks
   ✔ Q6: robustez/22 features: as médias das quatro arquiteturas ficam a menos de 1,5 p.p. umas das outras
   ✔ Q7: maior speedup na (256,256,256); speedup abaixo de 1 em todas as configurações e crescente com a arquitetura
   ✔ Q8: nas redes pequenas, (8,) e (64,64), a CPU foi mais rápida em todas as repetições, nas duas entradas
   ✔ Q8: leitura das faixas de repetições da Tabela B igual à citada (8 de 8 configurações com a CPU mais rápida em todas as repetições)
   ✔ Q8: da (8,) para a (256,256,256), o tempo da GPU cresce menos que o da CPU, nas duas entradas
   ✔ Q8: 10.2: na (256,256,256) o lote completo dá o maior speedup entre os lotes; na (8,) o speedup fica abaixo de 0,6 em todos os lotes
   ✔ Q8: 10.2: a (256,256,256) com lote completo fica entre 0,8 e 1,5 nas duas tabelas, com a leitura citada ("empate")
   ✔ Q8: 10.2: o speedup cresce com a largura, passa de 1,5 a partir de 512 neurônios e de 10 na (2048,2048,2048); da (256,256,256) para a (1024,1024,1024) o custo da GPU por época sobe menos de 1,6× e
o da CPU mais de 4×
   ✔ Q8: o 1º treino da GPU, ainda frio, demorou mais que a mediana aquecida da mesma configuração
   ✔ Q8: 10.3: dezenas de kernels por passo; na (8,) e na (64,64) a soma das durações dos kernels fica abaixo de 1/10 do tempo de parede do passo; a soma cresce da (8,) para a (256,256,256)
   ✔ Conclusão: com 22 features a (256,256,256) não acerta mais que a (8,) nem que a (64,64), em nenhum framework
(b) Números — assinatura das dependências das respostas: registrada f75e366e0320acdf (execução de 2026-10-14 08:16) | desta execução f75e366e0320acdf
   ✔ os números citados nas respostas são os desta execução
```

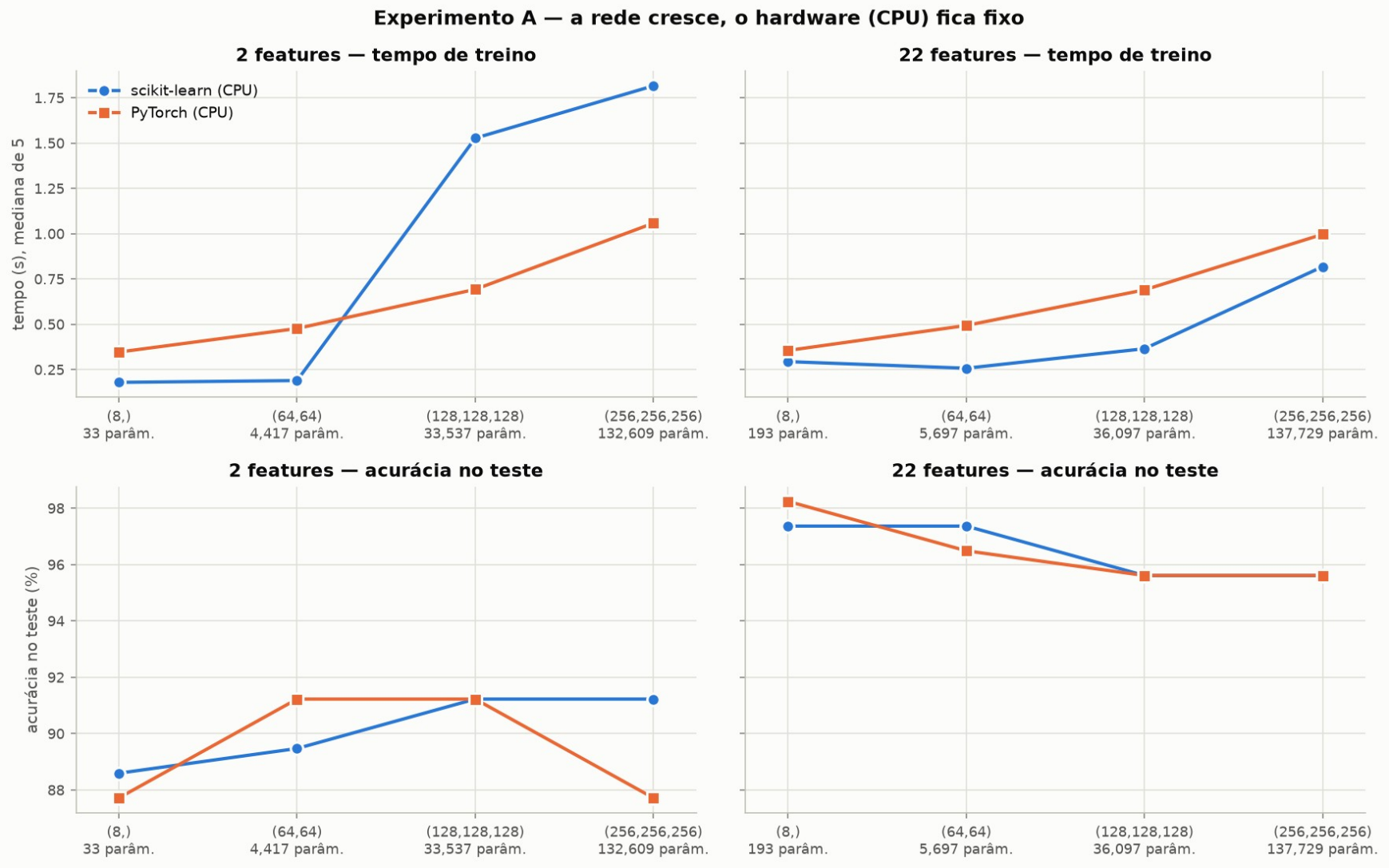


Figura 1 — Experimento A: tempo e acurácia por arquitetura (CPU).

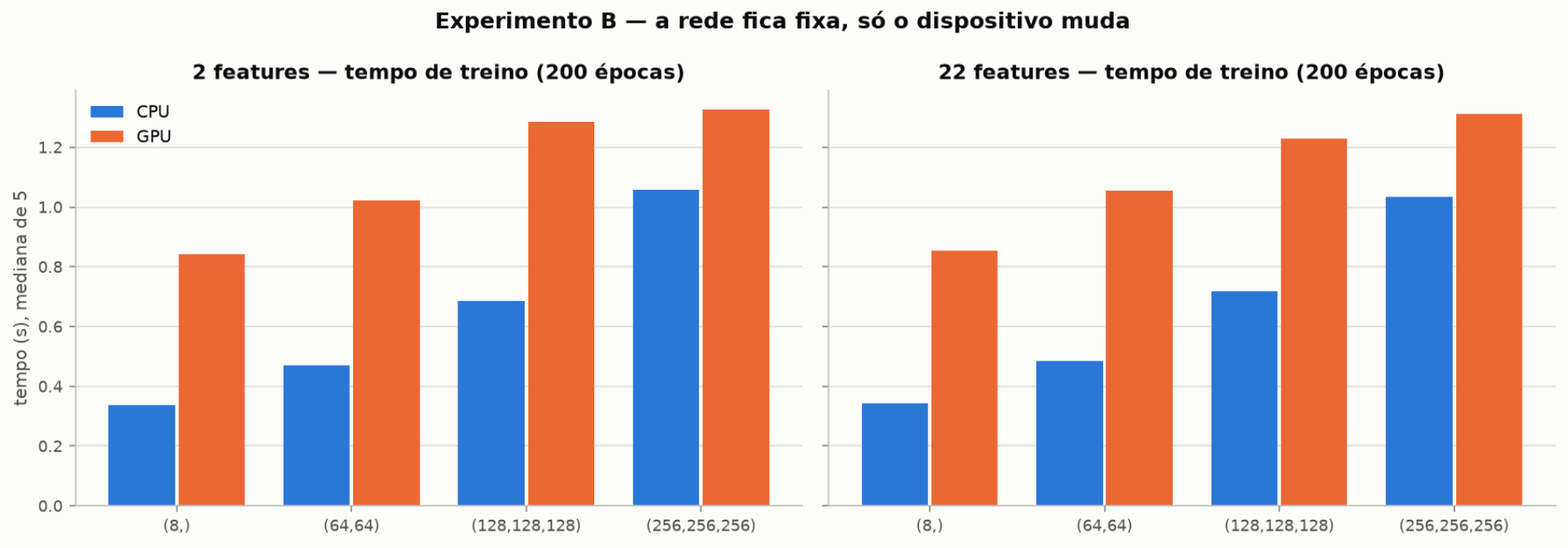


Figura 2 — Experimento B: tempo de treino na CPU e na GPU.

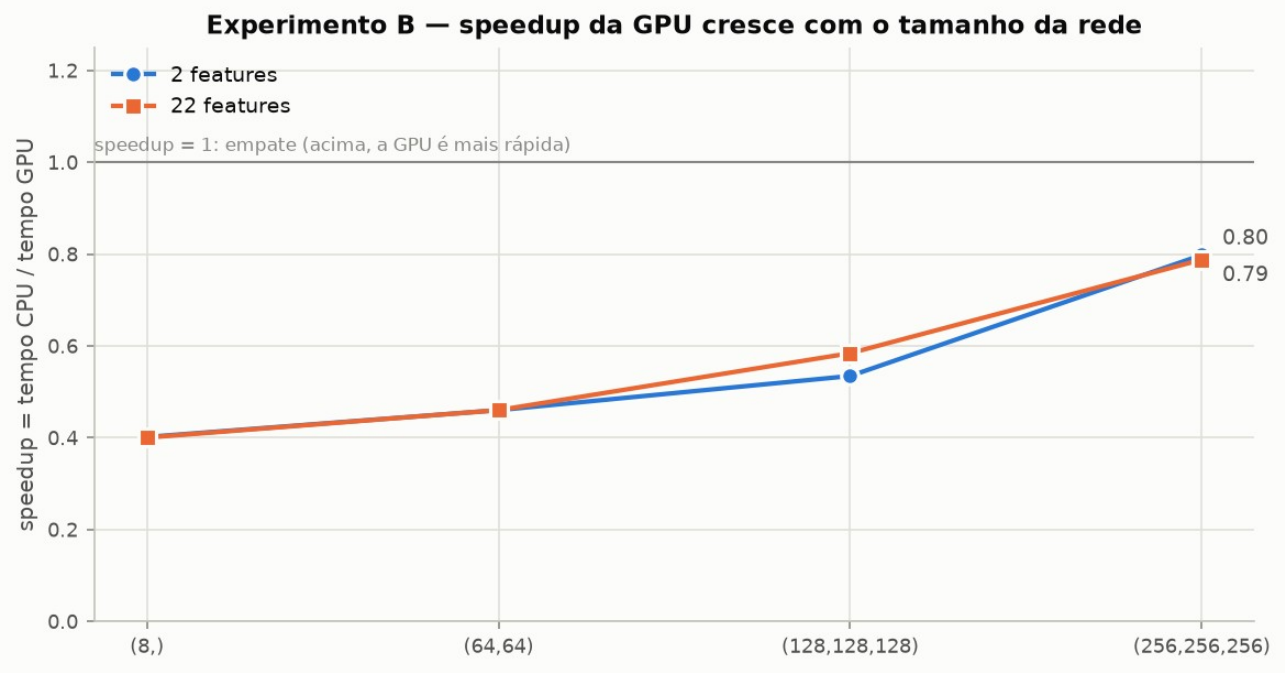


Figura 3 — Experimento B: speedup = tempo_CPU / tempo_GPU.

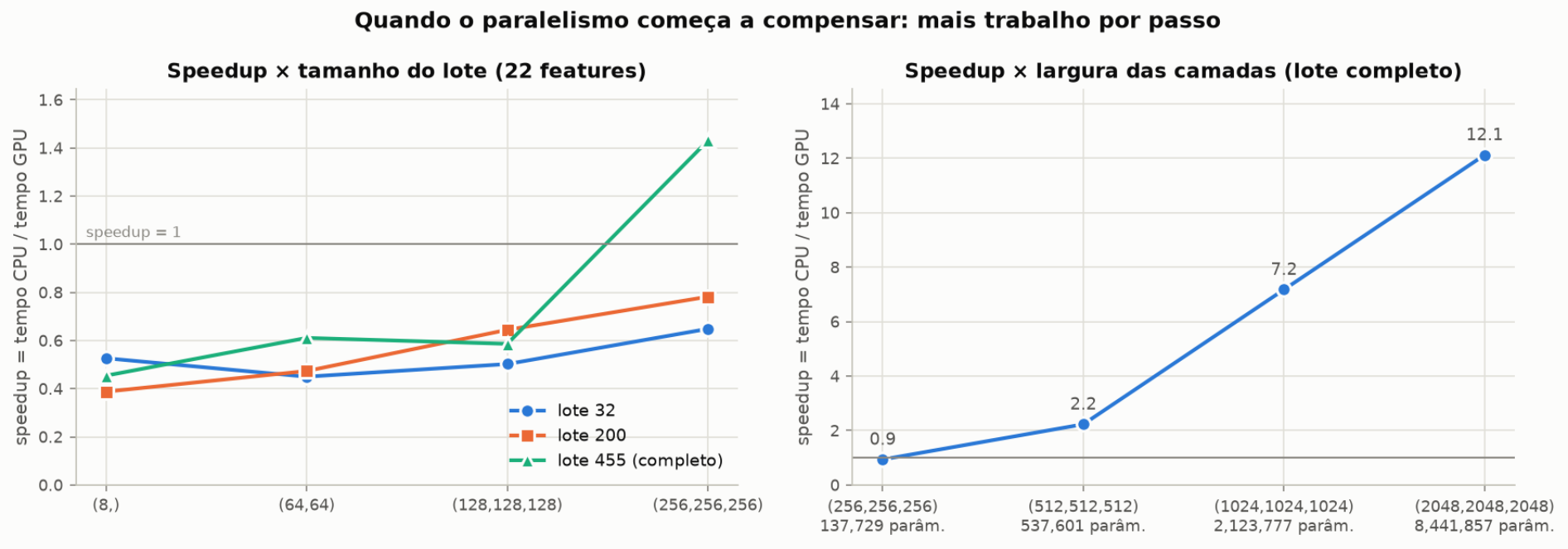


Figura 4 — Análise complementar: quando a GPU passa a compensar.